Lab-Bipolar-NRZ: Complete Explanation

Welcome! This is a fantastic lab to start with — it covers the *foundation* of all digital communication systems. Let me walk you through everything from scratch.

🧱 The Big Picture: What Are We Even Doing?

Imagine you want to send the message "Hello" over a wire or through the air. Computers store information as **bits** — 0s and 1s. The challenge is: *how do you physically send a 0 or a 1?* You need to convert those abstract numbers into a real, physical signal — a voltage, a radio wave, etc.

This lab is about the **simplest possible way** to do that, called **Bipolar NRZ (Non-Return-to-Zero)** signaling, and then studying what happens when that signal gets corrupted by noise.

📦 Page 1–2: The Block Diagram — The "Assembly Line" of Communication

Think of this like a postal system:

[You write a letter] → [Post office packages it] → [Truck drives through rain/traffic]
→ [Destination post office unpacks it] → [Recipient reads it]
In our system:

a[k] → p(t) → m(t) → +noise n(t) → r(t) → matched filter h(t) → y(t) → sample → â[k]

Let me explain each piece:

The Transmitter Side

a[k] — The Symbol Sequence

This is your binary data, but mapped to +1 and −1 instead of 0 and 1. Why ±1 instead of 0/1? Because a *symmetric* bipolar system is more noise-resistant (the two signal levels are as far apart as possible around zero). The mapping is simply:

- **bit 1 → symbol +1**
- **bit 0 → symbol −1**
-

The index *k* is **discrete** — it just counts which symbol we're on: ..., a[−1], a[0], a[1], a[2], ...

p(t) — The Transmit Pulse / Transmit Filter

Here's a key insight: you **can't just send a[k] directly — it's just a number**. You need a *physical waveform*. The transmit filter takes each symbol and "stamps" it onto a **pulse shape p(t).**

For the **k-th symbol**, the contribution to the transmitted signal is:

$$\mathbf{a[k] \cdot p(t-kT)}$$

This means: **take the pulse shape p(t), shift it to the k-th time slot (at time kT), and scale it by the symbol value a[k].**

m(t) — The Modulated Transmit Signal

$$m(t) = \sum_{k=-\infty}^{\infty} a[k] \cdot p(t - kT)$$

This is **Amplitude Shift Keying (ASK)** — the amplitude of the pulse encodes the symbol. The pulse shape p(t) is transmitted every T seconds (the **symbol period**), just with different amplitudes ±1.

Why the Dirac Impulse train?

The mathematical way to write this uses the Dirac delta δ(t):

$$m(t) = \left(\sum_{k=-\infty}^{\infty} a[k]\delta(t - kT) \right) * p(t)$$

Think of the Dirac impulse train as a "comb" — infinitely thin spikes at each symbol time, each weighted by a[k]. When you *convolve* this comb with the pulse p(t), each spike gets "replaced" by a copy of p(t) scaled by the symbol value. The identity used is:

Convolution with a shifted impulse = shifting the function. This is a fundamental property of convolution.

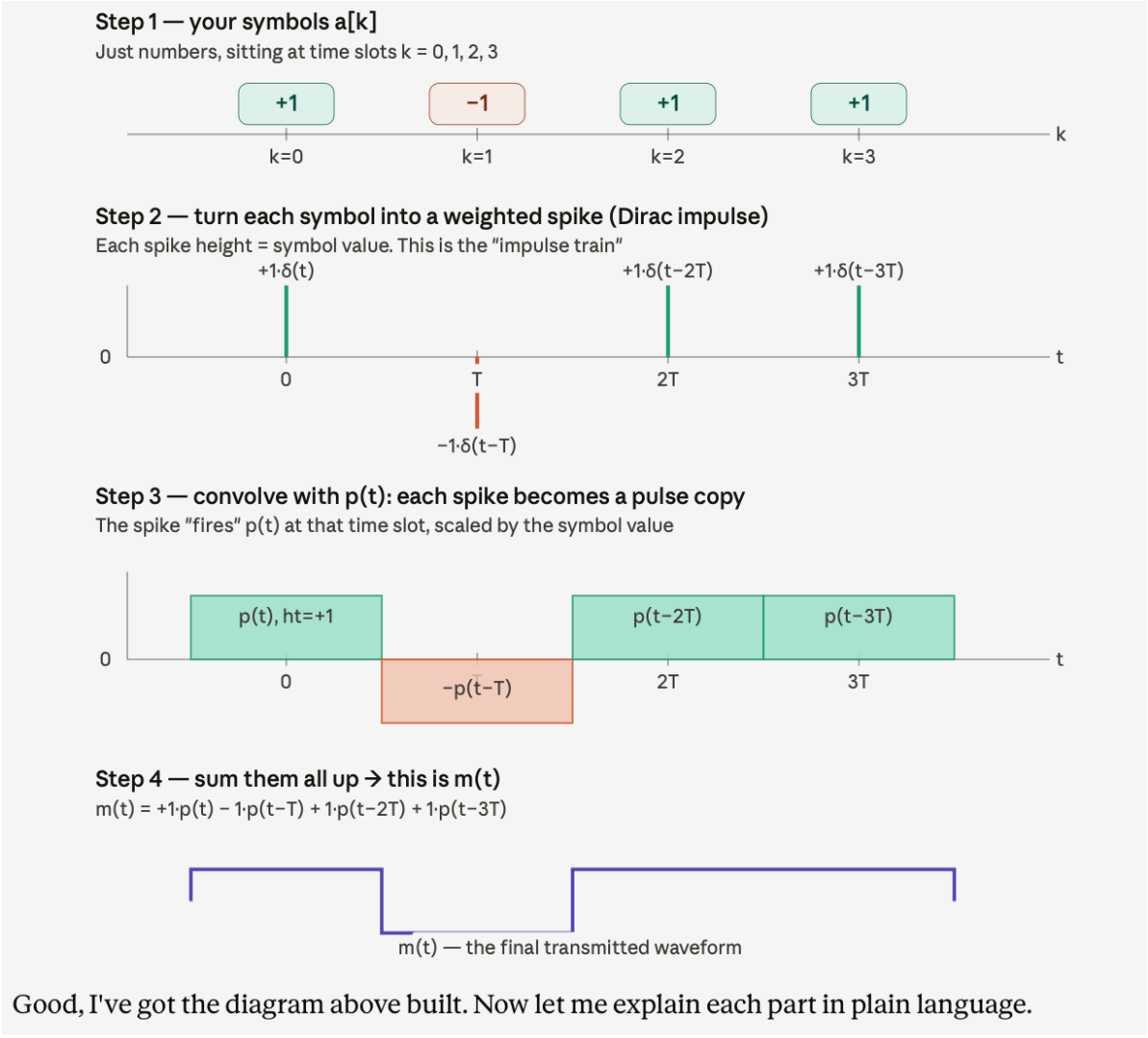
$$p(t) * \delta(t - kT) = p(t - kT)$$

Why Do We Need to "Stamp" a Pulse?

You have symbols: **a[k] = {+1, −1, +1, +1}**

These are just numbers sitting at discrete time indices k = 0, 1, 2, 3. They have no shape in time. To transmit them physically, you need to "stamp" each number onto a pulse shape p(t) and place it at the right moment in time.

Here's the idea visually — first, your symbols as isolated spikes:



Good, I've got the diagram above built. Now let me explain each part in plain language.

Why a Dirac Spike?

Think of it this way. You want to "mark" a specific moment in time — say, exactly at t = 0 — with a value of +1. But a normal signal has a *duration*. How do you mark a single, exact instant?

The Dirac impulse δ(t) is a mathematical idealization: infinitely tall, infinitely thin, but with an area of exactly 1. It exists only at one precise moment.

When you write **a[k] · δ(t – kT)**, you're placing a spike at time kT, with height a[k]. It's like stamping a dot on a timeline saying "*at this exact moment, the symbol has this value*."

The spike itself isn't what you transmit — it's the *instruction* for what to transmit.

What Is Convolution, Really?

Forget the integral formula for now. Here's the plain English version:

Convolution of a signal with p(t) means: wherever there's a spike, replace that spike with a copy of p(t) — scaled by the spike's height.

That's it. The spike says "*put a pulse here*." The convolution *does* the putting.

The mathematical identity that makes this work:

$$p(t) * \delta(t - kT) = p(t - kT)$$

Why use convolution and not just "place pulses manually"?

Because convolution is the universal mathematical operation for describing *what a filter does to a signal*. Every physical system — a wire, a filter circuit, an antenna — can be described by its impulse response. When you put a signal through that system, the output is always the convolution of the input with the impulse response.

So by writing $m(t) = a_{up}(t) * p(t)$, you're not just being fancy — you're describing the *real physical process* of the transmit filter shaping the symbols into a waveform.

The Channel

n(t) — AWGN (Additive White Gaussian Noise)

The signal travels through a medium (wire, air) and picks up noise. We model this as:

$$r(t) = m(t) + n(t)$$

"Additive" = noise simply adds to the signal (not multiplicative). "White" = noise has equal power at all frequencies (like white light). "Gaussian" = the noise amplitude at any moment follows a bell-curve (normal) distribution. This is the most common and mathematically tractable noise model.

The Receiver Side

h(t) = p(–t) — The Matched Filter

This is the clever part. The receiver uses a filter whose impulse response is the *time-reversed* version of the transmit pulse. Why? Because this choice **maximizes the Signal-to-Noise Ratio (SNR)** at the sampling instant — it's mathematically proven to be optimal for detecting signals in AWGN. The filter is "matched" to the transmit pulse, hence the name.

y(t) — Matched Filter Output

The received (noisy) signal r(t) is passed through h(t). The output y(t) is a "cleaned up" version where signal energy is concentrated at the optimal sampling instants.

y[k] = y(kT) — Sampling

We sample y(t) once per symbol period, at t = kT. These samples are called **soft symbols** — they're analog values close to ±1 (ideally) but perturbed by noise.

â[k] — Decision Device

A simple threshold comparator:

- If y[k] ≥ 0 → decide â[k] = 1 (symbol was +1)
- If y[k] < 0 → decide â[k] = 0 (symbol was −1)

Page 2–3: Discrete-Time Implementation (GNURadio)

Real computers can't process truly continuous signals. So everything gets **sampled** at a

rate $r_s = 1/T_s$ (samples per second). The key relationship is:

$$r_s = N \cdot r$$

where **r = 1/T is the symbol rate**
and **N is the number of samples per symbol (SPS)**.
In the flowgraph, N = **sps = 8**, meaning for every one symbol, there are 8 samples.

Upsampling

Since symbols arrive at rate r but we process at rate $r_s = N \cdot r$, we need to "stretch" the symbol sequence by inserting N−1 zeros between each symbol. This is called upsampling:

$$a_{up}[n] = \begin{cases} a[n/N] & \textbf{if } n/N \textbf{ is integer} \\ 0 & \textbf{otherwise} \end{cases}$$

So if a[k] = {+1, −1, +1, +1} and

N=4,

then: $a_{up}[n]$ = {+1, 0, 0, 0, −1, 0, 0, 0, +1, 0, 0, 0, +1, 0, 0, 0}

Then convolving $a_{up}[n]$ with the discrete pulse p[n] **"fills in" the zeros, producing the shaped transmit signal m[n].**

Why do we NOT use true impulses in discrete time?

Because in a computer or DSP system:

- signals are sampled
- time is indexed by integers n
- true Dirac impulses cannot exist digitally

A Dirac impulse has:

- infinite amplitude
- zero width

Impossible digitally.

So in discrete-time, we replace the impulse train with something equivalent.

Instead of:

$$\delta(t-kT)$$

we use:

$$\delta[n-kN]$$

which is just:

- 1 at one sample
- 0 everywhere else

That becomes the upsampled sequence.

What exactly is upsampling?

Suppose:

$$a[k] = [1,\ -1,\ \ 1]$$

and:

N = 4 (samples per symbol)

Then after upsampling:

$a_{up}\,[n]=[1,0,0,0,\ -1,0,0,0,\ 1,0,0,0]$

So we inserted: $N-1=3$, zeros between symbols.

This is the discrete-time equivalent of spacing impulses every symbol period.

Why do we insert zeros?

Because now the system operates at the higher sampling rate:

$r_s = Nr$

p[n] works sample-by-sample, not symbol-by-symbol.

The zeros create "space" for the pulse shape to develop between symbols.

Without upsampling:

- symbols would occur every sample
- no pulse shaping possible
- symbol timing would be wrong

What does the transmit filter do after upsampling?

The filter fills in those zeros with the pulse shape.

But right now only the FIRST sample contains energy. The other 3 are empty zeros.

So the signal is still not shaped properly. The transmit filter says:

"Don't keep the symbol energy at one instant. Spread it across the whole symbol duration."

For a rectangular pulse shape: p[n]=[1,1,1,1]

(ignoring normalization for simplicity)

This means:

- **one symbol should stay constant for 4 samples.**

Example:

Upsampled input: [1,0,0,0]

After rectangular pulse filter: [1,1,1,1]

The filter spreads one symbol over N samples.

That is pulse shaping. Because the filter "copies" the symbol into neighbouring samples.

So instead of:

- one spike at the beginning

we now have:

- a full pulse lasting 4 samples

Why is this necessary?

Because real communication systems cannot transmit isolated symbols instantaneously.

Every symbol must occupy time.

The transmit filter creates that time-domain waveform.

Without the filter, the signal would just be isolated spikes:

```
1  0  0  0 -1  0  0  0
```

which:

- **has terrible spectrum**
- **is not bandwidth efficient**
- **is not a proper pulse-shaped signal**

The MOST important intuition

- **Upsampling creates empty time slots.**
- **Transmit filter fills those slots with the pulse shape.**

Downsampling (Decimation)

After the matched filter, we only need one sample per symbol. So we take every N-th sample:

$$y[k] = y[n = k \cdot N]$$

Complete signal chain

Symbols
↓
Upsampling
↓
Transmit Filter
↓
Channel + Noise
↓
Matched Filter
↓
Downsampling
↓
Decision Device

1. Symbols at symbol rate

You begin with: **a[k]**

Example:

1 −1 1

This exists at the **SYMBOL RATE.**

Meaning:

- one value per symbol period

2. Upsampling happens BEFORE the transmit filter

This is VERY important.

The PDF says:

"The input signal $a_{up}[n]$ of the transmit filter is simply an upsampled version of a[k]

So:

a[k]

↓

Upsampler

↓

a_up[n]

Example:

Before upsampling:

1 −1 1

After upsampling with **N=4:**

1 0 0 0 −1 0 0 0 1 0 0 0

Now the signal runs at the **HIGHER SAMPLE RATE**

Why is upsampling needed BEFORE filtering?

Because the transmit filter operates sample-by-sample.

The filter needs:

- multiple samples per symbol
- space to create the pulse shape

Without upsampling:

- one symbol would last only one sample
- pulse shaping impossible

3. Transmit filter happens AFTER upsampling

The filter converts:

1 0 0 0

into:

1 1 1 1

(for a rectangular pulse)

Now the symbol occupies time.

This becomes the transmitted waveform: **m[n]**

4. Channel

Noise gets added: m[n] + noise

producing: r[n]

5. Matched filter

At the receiver:

r[n]
↓
Matched Filter
↓
y[n]

The matched filter:

- **collects energy**
- **improves SNR**
- **creates optimal sampling points**

BUT:

the signal is STILL at sample rate.

Meaning:

- ***many samples per symbol still exist***

6. Downsampling happens AFTER the matched filter

This is the key point. The PDF says:

"a corresponding downsampling operation … is performed as a decimation by a factor of N after the receive filter"

So:

Matched Filter Output y[n]
↓
Downsampler
↓
y[k]

What does the downsampler do?

It keeps only every N-th sample:

$$y[k] = y[n = kN]$$

Example with N=4:

Input:

0.1 0.5 0.8 1.0 -0.2 -0.7 -0.9 -1.0

Downsampler keeps:

1.0 -1.0

discarding the rest.

Now we are back at **SYMBOL RATE.**

Final stage: Decision device

Now the receiver decides:

```
If y[k] > 0  →  +1
If y[k] < 0  →  -1
```

recovering the transmitted bits.

Stage	Where it happens	What happens	Intuition	Mathematical view	Purpose
1. Symbol source	Transmitter input	Generates symbols $a[k]$ at symbol rate	One value per symbol	$a[k]$	Original information (bits mapped to symbols)
2. Upsampling	Before transmit filter	Insert $N-1$ zeros between symbols	"Create empty time slots"	$a_{up}[n]$ with zeros between samples	Increase rate from symbol rate → sample rate
3. Transmit filter (pulse shaping)	TX filter $p[n]$	Spreads each symbol over N samples	"Draw waveform from each symbol"	$m[n] = a_{up}[n] * p[n]$	Create physical waveform for transmission
4. Channel + noise	Physical channel	Adds noise/distortion	Signal gets corrupted	$r[n] = m[n] + n[n]$	Real-world transmission effects
5. Matched filter	RX filter $h[n] = p[-n]$	Maximizes SNR, aligns symbols	"Enhance correct sampling points"	$y[n] = r[n] * h[n]$	Improve detection quality
6. Downsampling (decimation)	After matched filter	Keep every N-th sample	"Pick best sample per symbol"	$y[k] = y[n = kN]$	Convert back to symbol rate
7. Decision device	Final receiver stage	Decide +1 or -1	"Make bit decision"	$\hat{b}[k]$ from sign of $y[k]$	Recover transmitted bits

Upsampling	**adds zeros → increases rate**
Pulse shaping	**makes waveform**
Matched filter	**maximizes SNR**
Downsampling	**selects 1 sample per symbol**

📐 Page 4: The GNURadio Flowgraph (Rectangular Pulse)

Block	Role
GLFSR Source	Generates pseudo-random bits (m-sequence) — our "data"
Map (0→−1, 1→+1)	Converts bits to bipolar symbols a[k]
Rational Resampler	Upsamples by factor N (inserts zeros)
Interpolating FIR Filter	Applies the rectangular transmit pulse p[n]
Noise Source + Add	Simulates the AWGN channel
Decimating FIR Filter	Applies matched filter h[n] = p[−n]
Keep 1 in N	Downsamples — takes the optimal sample
Threshold	Decision device: y[k] ⩾ 0 → 1, else → 0
BER	Counts bit errors by comparing Tx and Rx bits
QT GUI Time/Eye/Freq Sinks	Various signal visualizations

↓

Galois Linear Feedback Shift Register

The **rectangular pulse** has N non-zero values all equal to 1/√N. **The 1/√N normalization ensures the pulse energy equals 1:**

$$E_p = \sum_{n=-\infty}^{\infty} |p[n]|^2 = 1$$

In signals, **energy** means the same thing intuitively — how "strong" is this pulse overall? We calculate it by squaring each value of the pulse and adding them all up:
Our **rectangular pulse p[n] has N samples**, and **each sample has the value 1/√N.**
You have 4 samples, N = 4, each equal to 1/√4 = 0.5 The energy formula says — square each sample, then add them up.

$$E_p = \sum_{n} |p[n]|^2 = 0.5^2 + 0.5^2 + 0.5^2 + 0.5^2 = 1$$

So the energy E_p = **1** exactly. That's the whole point of choosing **1/√N as the sample value — it guarantees the energy always equals 1, no matter what N is.**

By **normalizing to $E_p = 1$**, everyone agrees on a common reference. It's like agreeing to use meters instead of random units.

The practical benefits are:

- Matched filter output is always clean **±1** at the sampling instant
- The decision threshold is always at **0** — exactly halfway between +1 and −1
- BER formulas become universal — they work for any pulse shape as long as energy is normalized
- When you change pulse shape (rectangle → RRCS), the BER comparison is **fair** because both have the same energy

Matched filter output at the best moment = **how well the signal matches the expected pulse** = **pulse energy E_p**
With $E_p = 1$ → output is ±1
With $E_p = 7$ → output is ±7

👁️ Page 5–7: Questions & The Eye Diagram

Answers to the Lab Questions (Rectangular Pulse Section)

Q1: What is the name of the samples per symbol parameter N? What value?

- The parameter is called **`sps` (Samples Per Symbol)**. It is set to **8**.

Q2: What is the sampling rate r_s?

- From the flowgraph: `samp_rate = 96k` (96,000 samples/second).
- The symbol rate is therefore r = **r_s / N =** 96000/8 = 12,000 symbols/second.

Q3: How are bits b[k] ∈ {0,1} mapped to symbols a[k] ∈ {±1}?

- Via the **Map block**: 0 → −1, 1 → +1. Mathematically: a[k] = 2·b[k] − 1.

The Eye Diagram — What Is It?

Imagine printing the signal y(t) on transparent paper, cutting it into segments of length 2T (two symbol periods), and stacking all the pieces on top of each other. What you see is the **eye diagram**.

- **Wide open eye** = low noise, no ISI → easy to make correct decisions
- **Closed eye** = heavy noise or ISI → many errors

At high SNR (40 dB), the eye is wide open and samples at $\mathbf{t = (N-1)\cdot T_s}$ hit exactly

$$\mathbf{\pm E_p = \pm 1.}$$

The Decision Rule (page 7)

$$\hat{b}[k] = \begin{cases} 1 & \textbf{if } y[k] \geq 0 \\ 0 & \textbf{if } y[k] < 0 \end{cases}$$

Why is threshold = 0?

Because transmitted symbols are symmetric:

Symbol
1
-1

The midpoint between them is: 0

WITHOUT noise:

Suppose transmitted symbol is: **+1**

After matched filtering and sampling: **y[k]=+1**

Receiver checks:

Is +1 ≥ 0 ? YES

So receiver decides: **b^[k]=1**

Correct.

Suppose transmitted symbol was: **−1**

Then: **y[k]=−1**

Receiver checks:

```
Is -1 < 0 ?
YES
```

So: **b^[k]=0**

Correct again

Now what happens WITH noise?

Noise changes the received sample.

Instead of receiving exactly: **+1**

we may receive: **0.7, 1.2, 0.4**

etc. Usually still positive. So decision remains correct.

But sometimes noise is very large

Suppose transmitted symbol was: **+1**

BUT noise is strongly negative.

Then received sample becomes: **y[k]=–0.3**

Now the receiver checks:

```
Is -0.3 ≥ 0 ?
NO
```

So receiver decides: **b^[k]=0**

which is WRONG

THIS is called crossing the threshold

The sample moved from the correct side of zero to the wrong side because of noise.

📉 Page 8: Noise Effects & BER

When SNR drops to 10 dB:

- The eye nearly closes
- y[k] deviates significantly from ±1
- BER is still **< 10^{-3} (less than 1 error per 1000 bits) — digital comms are very robust!**

At 7 dB: BER ≈ 10^{-2} At 0 dB: Eye diagram collapses, signal buried in noise.

Why is the rectangular pulse bad in frequency?

The rectangular pulse in time has a sinc (si) function spectrum in frequency:

$$P(f) \propto \text{sinc}(fT) = \frac{\sin(\pi f T)}{\pi f T}$$

The sinc function has large sidelobes — significant power far away from the main frequency. This causes out-of-band interference with neighboring channels. That's why rectangular pulses are only used in military systems (GPS etc.) where spectral efficiency isn't the priority.

🌊 Page 9–13: Root-Raised Cosine (RRCS) Pulse

Why RRCS?

We need a pulse that:

1. Is **bandwidth-limited** (no large sidelobes)
2. Has **zero ISI** at sampling instants

The **First Nyquist Criterion** is the key condition for zero ISI. It requires the autocorrelation of the pulse to satisfy:

$$\phi_{pp}^{E}[k \cdot N] = \begin{cases} E_p & k = 0 \\ 0 & k \neq 0 \end{cases}$$

The matched filter output is basically:

"How much does the received pulse match the expected pulse?"

the autocorrelation appears at the RECEIVER OUTPUT.

That "matching operation" mathematically becomes:

- **Autocorrelation.**

$$\phi_{pp}[n]$$

It measures:

"How much does the pulse overlap with a shifted version of itself?"

Mathematical definition

For discrete-time:

$$\phi_{pp}^{E}[n] = p[-n] * p[n]$$

This is exactly:

- **pulse convolved with time-reversed pulse**

which is ALSO what the matched filter does.

The matched filter output IS the pulse autocorrelation.

What does the Nyquist Criterion require?

At sampling instants: **n = kN**

we want:

```
Current symbol → nonzero contribution
Other symbols → exactly zero
```

So:

$$\phi_{pp}^{E}[kN] = 0 \quad \text{for } k \neq 0$$

Meaning physically

At symbol sampling times:

- neighboring pulses cross zero
- only the current pulse survives

NO ISI.

What pulse satisfies this?

The:

Raised Cosine Pulse

It is specially designed so that:

- zeros occur exactly at symbol intervals

Therefore:

- no ISI

10. Why use RRCS instead?

The actual system uses:

6. Why is $k = 0$ special?

Because:

$$k = 0$$

means:

- no shift
- pulse aligned with itself

So overlap is maximum.

That maximum value equals pulse energy:

$$E_p$$

Therefore:

$$\overset{E}{\varphi_{pp}}[0] = E_p$$

Usually normalized

Most systems normalize:

$$E_p = 1$$

So:

$$\overset{E}{\varphi_{pp}}[0] = 1$$

Caption

Root Raised Cosine (RRCS)

because:

- TX filter uses one square root
- RX matched filter uses the other square root

Together: RRCS. × RRCS =. Raised Cosine

So the COMBINED response satisfies Nyquist.

Quantity	Meaning
$p[n]$	transmit pulse
$h[n] = p[-n]$	matched filter
$\varphi_{pp}[n]$	autocorrelation of pulse
Where appears?	after matched filter
Why important?	determines ISI
Nyquist criterion	neighboring samples must be zero

What is the rolloff factor α ?

The rolloff factor controls:

how much EXTRA bandwidth the raised cosine pulse uses beyond the minimum required bandwidth.

Small α

narrow bandwidth
sharp edges
long ringing in time

Large α

wider bandwidth
smoother transitions
shorter ringing

Why is it called “rolloff”?

Because the spectrum “rolls off” gradually instead of cutting sharply.

The RRCS transmit filter **p[n]** and the corresponding matched-filter **h[n] = p[−n]** both introduce a **transmission delay** that needs to be compensated for by the Delay block through which the transmit bits b[k] ∈ {0, 1} are piped before being displayed together with the received bits ^ b[k] in the QT GUI Time Sink.

The lab asks:

Which value of the Delay parameter perfectly compensates the delay introduced by the filters?
To the value of which other top-level parameter does this correspond?

The Delay parameter has to be adjusted until the transmitted and received bit sequences overlap.
The correct Delay value corresponds to the top-level parameter **nsymbols_rrcs,** which describes the length of the RRC filter impulse response in symbol periods.

Delay = 50
It corresponds to nsymbols_rrcs = 50.

Page 11: RRC with resampling inside filters

In the previous RRC flowgraph, upsampling and downsampling were done using separate blocks.

In this flowgraph, th**e RRC filters themselves do the resampling.**

So the system is more efficient.

Simple explanation:

This flowgraph combines pulse shaping and resampling inside the RRC filters.
The resulting signals are essentially the same, but the flowgraph is computationally more efficient.

For RRC with resampling at SNR = 10 dB:
log10(BER) = _____
BER = 10^(_____) = _____

Compare with rectangular pulse

The lab asks:

`Compare this BER to the BER for SNR = 10 dB with the rectangular transmit pulse. What do you find?`

Expected answer:

The BER values are approximately the same.

Why?

Because both systems are designed without intersymbol interference and use matched filtering. If the pulse satisfies the Nyquist criterion and the energy is normalized, the BER mainly depends on SNR, not directly on the pulse shape.

The BER for the RRC pulse is approximately the same as for the rectangular pulse at the same SNR.

Small differences may occur because of random noise and finite simulation length.

This shows that when the pulse shape satisfies the Nyquist criterion and the filters are matched and energy-normalized, the BER mainly depends on SNR rather than the pulse shape.

No — if the Nyquist criterion is fulfilled. Here's why: the BER of a bipolar system in AWGN with a matched filter depends only on the energy per bit E_p and the noise power spectral density N_0:

$$\text{BER} = Q\left(\sqrt{\frac{2E_p}{N_0}}\right)$$

where Q(·) is the Q-function (tail probability of the Gaussian). Since both rectangular and RRCS pulses are **normalized to E_p = 1,** and both **use matched filters, t**he BER is **identical** ,as long as there's no ISI (Nyquist criterion satisfied). The simulation should confirm this: the BER at SNR = 10 dB should be the same for both pulse shapes.

🎓 Conceptual Questions

Q1. Why do we use ±1 instead of 0/1 for bipolar signaling?

If we used 0 and 1, the two signal levels are **not symmetric around zero**. The decision threshold would need to be at 0.5, and the distance from each level to the threshold is only 0.5.

With ±1, both levels are **equally far from zero** — distance of 1 on each side. This means:

- The threshold stays at 0 always
- The gap between the two levels is **2** instead of 1 — twice as large
- Larger gap means noise has to be twice as large to cause an error
- So the system is **more resistant to noise**

Also mathematically, the average transmitted power with 0/1 is not zero — there's a DC component. With ±1, the average is zero — no wasted DC power.

Q2. What does the matched filter do and why does it maximize SNR?

The matched filter is a filter in the receiver whose impulse response is the **time-reversed version of the transmit pulse**: $h(t) = p(-t)$.

What it does in plain English: it asks "how much does the received signal **look like** the pulse I was expecting?" It does this by correlating the received signal with the expected pulse shape.

Why it maximizes SNR: at the sampling instant, it adds up all the signal energy **coherently** — every sample of the signal contributes positively. But the noise adds up **randomly** — some positive, some negative, so they partially cancel. The result is that signal is as large as possible relative to noise — maximum SNR.

It's like asking multiple witnesses the same question , their true answers reinforce each other, but their random errors cancel out.

Q3. What is the difference between a soft symbol y[k] and a hard symbol â[k]?

A **soft symbol** y[k] is the **analog output** of the matched filter, sampled at the symbol rate. It's a continuous value — could be 0.87, –1.23, 0.45, etc. It contains information about *how confident* we are — a value of +0.99 means very likely a +1 was sent, while +0.1 means barely above threshold.

A **hard symbol** â[k] is the **binary decision** made by the threshold comparator:

- $y[k] \geq 0 \rightarrow \hat{a}[k] = 1$
- $y[k] < 0 \rightarrow \hat{a}[k] = 0$

Once you make the hard decision, all the confidence information is thrown away. You just get a 0 or 1.

Q4. What does "Non-Return-to-Zero" mean?

It refers to what the signal does **between symbols**.

In **Return-to-Zero (RZ)** signaling, the signal returns to zero voltage in the middle of every symbol period , even if the same symbol is repeated. So every bit has a **"reset."**

In **Non-Return-to-Zero (NRZ)** signaling, the signal **stays at its level** for the entire symbol period without returning to zero. If you send +1 followed by another +1, the signal just stays at +1 — it never dips back to zero between them.

NRZ is more spectrally efficient because it doesn't waste half the symbol period going to zero.

🔢 Mathematical Questions

Q5. If sps = 8 and samp_rate = 96 kHz, what is the symbol rate?

The relationship is: $r_s = N \cdot r$

Where $\mathbf{r_s}$ **= sampling rate, N = sps** = 8, **r = symbol rate.**

Solving for r:

$$r = \frac{r_s}{N} = \frac{96000}{8} = 12000 \text{ symbols/second}$$

The symbol period is therefore: $T = \frac{1}{r} = \frac{1}{12000} \approx 83.3\,\mu s$

Q6. Prove that rectangular p[n] with N values of 1/√N has unit energy.

The energy of a discrete signal is defined as: $E_p = \sum_{n=-\infty}^{\infty} |p[n]|^2$

The rectangular pulse has exactly N non-zero samples, each equal to 1/√N. All other samples are zero.

So: $E_p = N\left[\left(\frac{1}{N}\right)^2 + \left(\frac{1}{N}\right)^2 + \cdots\right]$ …N times

$$= N \cdot \left(\frac{1}{\sqrt{N}}\right)^2 = N \cdot \frac{1}{N} = 1\checkmark$$

Q7. What is p(t) ∗ δ(t – kT)? Derive it.

The convolution of p(t) with δ(t – kT) is defined as:

$$p(t) * \delta(t - kT) = \int_{-\infty}^{\infty} p(\tau) \cdot \delta(t - kT - \tau)\, d\tau$$

The Dirac delta δ(t – kT – τ) is zero everywhere **except** when its argument equals zero:

$$t - kT - \tau = 0 \Longrightarrow \tau = t - kT$$

So the integral only picks up the value of p(τ) at τ = t – kT:

$$= p(t - kT)$$

In plain English: the delta function acts as a **sampling/shifting device,** it "picks out" p(t) evaluated at the shifted location. The result is p(t) delayed by kT.

🏗️ System Design Questions

Q8. Why can't you use a rectangular pulse in commercial LTE/5G?

The rectangular pulse has a **sinc-shaped spectrum** in the frequency domain:

$$P(f) \propto \text{sinc}(fT)$$

The sinc function has very large **side lobes** — significant signal energy far away from the main frequency band. These sidelobes cause **out-of-band emissions** that interfere with signals in neighbouring frequency channels.

In commercial systems like LTE/5G, spectrum is extremely valuable and strictly regulated. You cannot interfere with neighbouring channels. Therefore you need a pulse shape with a **compact, controlled spectrum** — like the root-raised cosine — which confines almost all energy within a defined bandwidth.

The rectangular pulse is only acceptable in military systems like GPS where spectral coexistence is less critical.

Q9. Why must both transmitter and receiver use the RRCS filter, rather than one using the full raised cosine?

The goal is that the **combined response** of transmitter and receiver equals the raised cosine, which satisfies the Nyquist ISI-free criterion.

If we call the transmit filter P(f) and receive filter H(f), the combined response is:

$P(f) \cdot H(f) =$ Raised Cosine

For the matched filter condition, we need H(f) = P*(f) (the conjugate of P(f)). For real symmetric filters this means H(f) = P(f).

So both filters must be **identical** and each must be the **square root** of the raised cosine — the RRCS. Together they multiply to give the full raised cosine.

If only one filter had the full raised cosine and the other had nothing, the matched filter condition would be violated, you'd get suboptimal SNR.

Q10. If you double the symbol rate but keep the same noise level, what happens to BER?

Doubling the symbol rate means halving the symbol period T. To maintain the same pulse energy with a shorter pulse, you'd need to increase the pulse amplitude. But if noise level stays the same, the SNR **per symbol** decreases.

More practically, **doubling the symbol rate doubles the bandwidth required.**

If the noise power spectral density N_0 stays the same but bandwidth doubles, the **total noise power increases**. The SNR drops, and therefore the **BER increases,** more errors.

This is the fundamental tradeoff in communications: higher data rate requires more bandwidth, which admits more noise.

👁️ Eye Diagram Questions

Q11. You observe a partially closed eye diagram. What are the two possible causes?

Cause 1 — Noise: Random noise perturbs the signal at the sampling instant, spreading the signal trajectories up and down. The eye closes vertically, **the gap between the +1 and −1 levels shrinks.**

Cause 2 — Intersymbol Interference (ISI): If the pulse shape does not satisfy the Nyquist criterion, energy from one symbol "leaks" into adjacent symbol periods. This causes the eye

to close **both vertically and horizontally,** the crossing points become smeared and the opening shrinks.

You can distinguish them: noise causes **random fuzzy spread**, while ISI causes **deterministic systematic closing** — specific trajectories that don't belong at those positions.

Q12. Where in the eye diagram do you ideally want to place your sampling instant, and why?

At the point of **maximum eye opening** — where the vertical gap between the upper and lower trajectories is largest.

This is the point where:

- The signal is furthest from the threshold → largest noise margin
- The probability of noise pushing the signal across the threshold is minimum
- Therefore BER is minimized

For the **rectangular pulse** with N=8 samples, this is at sample index **N−1 = 7**. For the **RRCS** pulse it is similarly at the **center of the symbol period where the autocorrelation peaks.**

📉 BER Questions

Q13. If $\log_{10}$(BER) ≈ −3, what is the actual BER value?

$$\log_{10}(\text{BER}) = -3$$

$$\text{BER} = 10^{-3} = 0.001$$

This means 1 bit error per 1000 transmitted bits. In a system transmitting 12,000 symbols/second, that's about 12 errors per second , rare enough that you'd barely notice in a voice call, but significant for data.

Q14. Why is the BER the same for RRCS and rectangular pulses at the same SNR?

Because **BER d**epends only on two things:

1. The **energy per symbol** E_p

2. The **noise power** N_0

The formula is: $\text{BER} = Q\left(\sqrt{\frac{2E_p}{N_0}}\right)$

Both **rectangular and RRCS pulses are normalized to E_p = 1.**

Both use matched filters. Both satisfy the Nyquist criterion, so there is no ISI. The only thing that determines BER is the **SNR**, which is the same for both.

The pulse shape affects **bandwidth and spectral efficiency** , but not BER, as long as Nyquist is satisfied and energy is normalized.

Q15. What happens to BER if you sample at a non-optimal time instant?

The BER **increases,** potentially dramatically.

At the optimal sampling instant, the signal value is at its maximum **$\pm E_p$ = ±1,** giving the **largest possible distance from the threshold.** At any other instant, the signal value is smaller , closer to zero , meaning noise doesn't need to be as large to push it across the threshold.

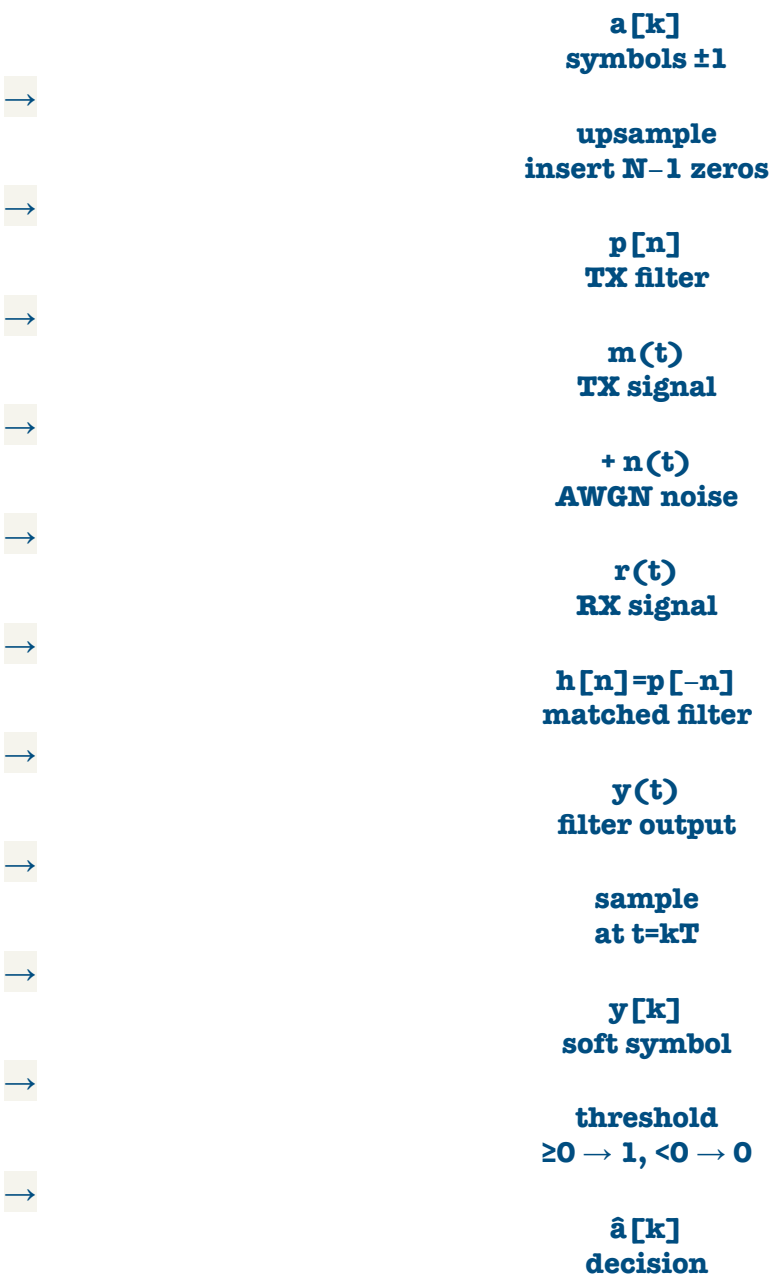
Additionally, at non-optimal instants, contributions from neighboring symbols may not be zero, meaning ISI enters the picture even if the pulse theoretically has no ISI at the correct sampling point.

In the worst case, sampling at exactly the wrong moment could make the eye completely closed, BER approaches 0.5, which means you're essentially **guessing randomly**. No better than flipping a coin.

SUMMARY

🔁 The system — what's always happening in all 3 parts

Every part of this lab is built on the same communication chain. Only the pulse shape changes.



ALL SIGNALS DEFINED

a[k] ∈ {±1}

Transmitted symbols. Bits mapped: 0→−1, 1→+1. Just numbers, no shape.

p[n] — transmit pulse

The shape stamped onto each symbol. Template used by TX filter. Energy $E_p = 1$.

m(t) — TX signal

Sum of all pulse copies: m(t) = Σ a[k]·p(t−kT). What goes into the channel.

n(t) — AWGN noise

Additive White Gaussian Noise. Random. Models real-world channel impairments.

r(t) = m(t) + n(t)

Received signal. Noisy version of m(t). Input to the receiver.

h[n] = p[−n]

Matched filter. Time-reversed pulse. Maximises SNR at sampling instant.

y(t) — filter output
Matched filter output. Continuous signal. "Cleaned up" version of r(t).
y[k] — soft symbol
y(t) sampled at t=kT. Analog value ≈ ±1. Contains confidence information.
â[k] — hard decision
Binary output of threshold device. 0 or 1. Confidence info is lost here.
aup[n] — upsampled
Symbols with N–1 zeros inserted between each one. Input to TX filter.

KEY NUMBERS (FROM FLOWGRAPH)

sps = N = 8
Samples per symbol. Upsampling factor. Discrete-time symbol period.

samp_rate = 96 kHz
$\mathbf{r_s = N \times r}$ = 8 × 12000. Sampling rate of the system.

symbol rate = 12 kHz
$\mathbf{r = r_s / N}$ = 96000 / 8. One symbol every T = 1/12000 s.

optimal sample: n = N–1 = 7
Where matched filter output peaks. Maximum eye opening. Minimum BER.

Part 1 — BipolarNRZrect.grc

Rectangular pulse shape

The simplest possible pulse — a flat rectangle. Goal: understand the basic system before worrying about spectral efficiency.

WHAT THE PULSE LOOKS LIKE

N = 8 samples, each equal to $1/\sqrt{N} = 1/\sqrt{8} \approx 0.354$. Flat top, sharp edges.

$p[n] = 1/\sqrt{N}$ for n = 0,1,...,N–1 | $E_p = N \times (1/\sqrt{N})^2 = 1$ ✓

WHY ENERGY = 1?

We normalise so every system uses the same reference. Then matched filter output = $\pm E_p = \pm 1$, threshold stays at 0, and BER formula works universally.

EYE DIAGRAM OBSERVATIONS

SNR = 40 dB (high)
Eye wide open. Samples hit exactly ±1. TX and RX bits identical. Optimal sample at n = N–1 = 7.
SNR = 10 dB (medium)

Eye nearly closed. y[k] deviates from ±1. BER ≈ 10^{-3} (1 error per 1000 bits). Still works!
SNR = 7 dB
First visible bit errors. BER ≈ 10^{-2}.
SNR = 0 dB
Eye completely closed. Signal buried in noise. System fails.

THE BIG PROBLEM WITH RECTANGULAR PULSES

Frequency spectrum = sinc function → huge sidelobes → out-of-band interference with neighbouring channels → NOT usable in commercial systems (LTE, 5G). Only used in military (GPS).

DECISION RULE

â[k] = 1 if y[k] ≥ 0 | â[k] = 0 if y[k] < 0

Part 2 — BipolarNRZ.grc

Root-Raised Cosine (RRCS) pulse shape

Fix the rectangular pulse's spectral problem. Goal: limited bandwidth AND zero ISI.

WHY RRCS?

Problem with rect pulse
Sinc spectrum → large sidelobes → interferes with neighbours.

RRCS solution
Compact spectrum. Almost all energy within defined bandwidth. No sidelobes.

THE NYQUIST CRITERION — WHY THERE'S NO ISI

The autocorrelation of p[n] with itself must have zeros at all symbol times except k=0:

$\phi pp[k \cdot N] = E_p$ if k=0 | $\phi pp[k \cdot N] = 0$ if k≠0

This means: at every sampling instant, only the current symbol contributes. Past and future symbols contribute exactly zero. No ISI.

WHY TX AND RX BOTH USE RRCS (NOT FULL RAISED COSINE)?

Combined response = TX filter × RX filter = raised cosine. So each must be the square root → RRCS. Also, RX filter must be matched to TX pulse → h[n] = p[–n] = p[n] (RRCS is symmetric). Both conditions satisfied only if both use RRCS.

THE DELAY PROBLEM

RRCS has many taps → introduces a transmission delay. Must compensate with a Delay block. Correct delay = nsymbols_rrcs × sps / 2 (half the filter length).

ALPHA (ROLLOFF FACTOR) = 0.35

α = 0

Minimum bandwidth. Brick-wall filter. Impossible to implement perfectly.

α = 1

Double bandwidth. Smooth. Easy to implement. Wastes spectrum.

α = 0.35 is the practical compromise used in real systems.

BER COMPARISON TO RECTANGULAR

BER is IDENTICAL to rectangular at the same SNR. Why? BER = $Q(\sqrt{(2E_p/N_0)})$. Both pulses have $E_p = 1$. Both use matched filter. Both satisfy Nyquist (no ISI). So BER depends only on SNR — not pulse shape.

Part 3 — BipolarResamplingInRRCS.grc

Resampling inside the RRCS filters

Efficiency improvement. Goal: reduce computational overhead by combining upsampling/downsampling with the filter itself.

WHAT CHANGES?

Part 2 approach

Separate blocks: Rational Resampler (upsample) → RRCS filter → Keep 1 in N (downsample). More blocks = more context switches = slower.

Part 3 approach

RRCS filter does upsampling AND filtering in one block. Same for RX. Fewer blocks = faster. Same output signals.

WHAT YOU LOSE

The eye diagram can no longer be displayed. Why? The matched filter output y[n] at sample rate is no longer a separate signal — it's internal to the combined block. You only see y[k] at symbol rate after decimation.

DOWNSAMPLING BEHAVIOUR

Takes the first of every N samples, discards N−1. This works because the filter delay is an integer multiple of N — so the first sample after decimation is already at the optimal sampling point.

REAL-WORLD LIMITATION HIGHLIGHTED

In a real receiver, the optimal sampling point is NOT known in advance — it could be any of the N samples per symbol period. Timing synchronisation is needed. That's a topic beyond this lab.

SNR ALREADY SET TO 10 DB HERE — COMPARE BER

$\log_{10}(BER) \approx -3 \rightarrow BER = 10^{-3} = 0.001$ (same as Part 1 ✓)

📌 The one big takeaway from the whole lab

BER depends only on E_p/N_0 (energy per symbol / noise), NOT on pulse shape — as long as Nyquist is satisfied and energy is normalised. Pulse shape affects bandwidth and spectral efficiency. **BER is determined purely by SNR.**

$BER = Q(\sqrt{(2E_p/N_0)})$ — universal for all Nyquist-satisfying bipolar systems

---